

**Singapore Institute of Technology
Information and Communications Technology Cluster**

ICT4011 Capstone Project

Capstone Project Progress Report Submission Form (Form E1)

A. Preamble

This form duly filled and signed must be attached to each Capstone Project Progress Report submitted.

Reporting Period:

(Reporting Period): 5/5/2025 to 10/8/2025

Candidate Particulars:

(Name of Candidate): Tay Siang Long

(Organization): ST Engineering Urban Solutions Ltd.

(Project Period): 5/5/2025 to 10/4/2026

Main Academic Supervisor Details:

(Name): Spiridon Bakiras

(Email Address): spiridon.bakiras@singaporetech.edu.sg

(Contact Number): 65923324

(Designation): Associate Professor

Industry Supervisor Details:

(Name): Wu Yingjie

(Email Address): yingjie.wu@stengg.com

(Contact Number): 88517312

(Designation): Assistant Principal Engineer

(Department / Division): URS

B. Declaration of Conformity

B.1 Declaration by Industry Supervisor

I hereby acknowledge that I have read and understood the contents of the Capstone Project Progress Report, and deem the contents appropriate for release to the Singapore Institute of Technology for use in the assessment of the candidate.

Signature



Name of Industry Supervisor: Wu Yingjie
Designation: Assistant Principal Engineer
Department: URS
Contact No: 88517312
Email: yingjie.wu@stengg.com
Date: 31/7/2025

B.2 Declaration by Candidate

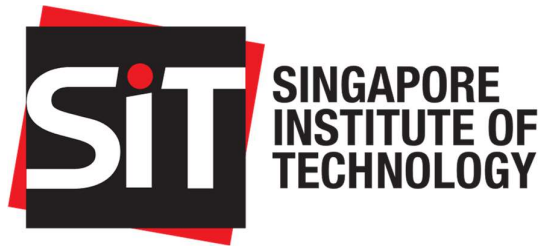
I hereby acknowledge that I have engaged and discussed with my Industry Supervisor on the contents of the Capstone Project Progress Report, and have sought approval for the release of the report to the Singapore Institute of Technology.

Signature



Name of Candidate: Tay Siang Long
Contact No: 81618762
Email: 2203190@sit.singaporetech.edu.sg
Date:

END OF FORM E



Information and Communications Technology Cluster

Multi-Agent System for Query-to-Dashboard

**Capstone Project Progress Report
(ST Engineering Urban Solutions Ltd.)**

For the reporting period from 5/5/2025 to 10/8/2025

**Tay Siang Long
2203190**

Name of Industry Supervisor: Wu YingJie

Name of Academic Supervisor: Spiridon Bakiras

Submitted as part of the requirement for ICT4011 Capstone Project

(Body Text, 16, Bold, Center) Capstone Project Progress Report for the Period of 5/5/2025, to 10/8/2025

1. Introduction

This project aims to automate the transformation of natural language queries into structured analytical reports and dashboards using a modular multi-agent system. The core component is the LangGraph React Analysis Agent, which leverages modern LLMs and a Model Context Protocol (MCP) server for dynamic tool orchestration.

2. Literature Review

ReAct Agents and LangGraph for Automated Analysis

ReAct-style agents—short for “Reasoning and Acting”—are designed to let language models solve complex tasks by interleaving step-by-step reasoning with tool use. Instead of answering a query outright, the agent thinks through the problem, calls tools (like a statistical function or plot generator), observes the result, and repeats this loop until a final answer is formed [1]. This method improves interpretability and accuracy, especially in domains like data science where numerical reasoning and visualization are key.

LangGraph, an extension of LangChain, provides a framework for building these kinds of ReAct agents. It gives developers a structure to define tool calls, handle state, and manage loops in the agent's reasoning cycle [2]. For example, LangChain's LangGraph ReAct agent template allows a language model to dynamically plan, invoke tools like matplotlib or pandas-based functions, and render intermediate outputs as part of a workflow [2][3]. These capabilities align closely with the design of our LangGraph ReAct Analysis Agent, which interprets natural language queries and uses backend tools to generate both reports and dashboards.

Recent case studies, such as Ionio's GitHub projects and documentation, show how ReAct agents can be used in enterprise settings to automate dashboard creation, handle real-time insights, and embed statistical pipelines [4][5]. These implementations inspired the design of our agent, particularly in terms of its looped reasoning flow, report generation, and multi-stage visual output.

Model Context Protocol (MCP) as a Tool Registry

The Model Context Protocol (MCP) is a relatively new but powerful concept introduced to standardize how agents and language models interact with external tools. Rather than hardwiring tools into the agent, MCP servers expose tools (like statistical functions or visualizers) through a registry that agents can query and invoke dynamically [6].

Anthropic and Microsoft have championed MCP as a way to enable models like Claude or Copilot to call enterprise-grade tools safely and flexibly. The protocol defines tools via OpenAPI schemas and allows real-time discovery without needing the agent to be redeployed [6][7]. In our project, this is implemented using a FastMCP server, which registers Python tools for data cleaning, anomaly detection, and distribution analysis. The analysis agent connects to this server, pulls available tools, and uses them on demand depending on the query—mirroring real-world usage of MCP by industrial systems.

GitHub repositories like MCP-Server-Data-Exploration and articles from ArsTurn also show practical implementations of MCP used in real-time analytics workflows [8][9]. These helped guide how tools were structured and registered in our own MCP setup.

Agent-to-Agent (A2A) Protocols for Multi-Agent Collaboration

Multi-agent systems require consistent, structured message format for communication. The Agent-to-Agent (A2A) protocol, introduced in 2025 and supported by organizations like Google and the Linux Foundation, defines how agents send each other messages, results, or task assignments [10].

A2A uses standard HTTP+JSON messaging with unique task IDs, error handling, and data artifact sharing. It enables agents built using different frameworks (LangChain, AutoGen, or custom frameworks) to interoperate without tight coupling [10][11]. In our project, a simplified version of A2A is used to enable structured exchanges between the analysis agent and other agents like orchestrator agent. Messages follow a dataclass format, tracking queries, results, and feedback within the pipeline.

Application to the Capstone System

Together, ReAct, MCP, and A2A create the foundation for a scalable, modular query-to-dashboard pipeline. The ReAct agent interprets user queries, selects tools from an MCP server, runs the analysis, and returns visual reports. If extended further, A2A can allow the agent to delegate tasks to others, such as a dashboard renderer or alert generator.

Recent literature, including Ran Zhang et al.'s paper on multi-agent visualization systems, emphasizes the value of these designs in enterprise analytics: systems that generate dashboards aren't just about displaying data—they structure narratives around insights and automate decision-making [12]. Our architecture is aligned with that goal, making use of state-of-the-art methods to deliver automated, explainable analytics for ST Engineering's transport datasets.

3. Planned Tasks and Objectives

The plan for this project was to develop a quantitative analysis agent with strong statistical and mathematical capabilities. The agent was intended to automatically determine which analysis function best fits each incoming query—whether it's a comparison, trend detection, anomaly identification, or distribution analysis. To support this, the system would use a Model Context Protocol (MCP) server as a centralized library of reusable analysis functions, covering common query types and use cases. This also included basic data cleaning and preparation steps to ensure reliable results.

A key goal was to make the agent modular, flexible, and scalable. The architecture was designed around a class structure, making it easy to add new features and tools as requirements evolve. For tool discovery and execution, the agent would connect to the FastMCP server, enabling real-time retrieval and invocation of data science tools for workflows such as exploratory data analysis (EDA), clustering, and statistical testing.

Visualization was another important aspect. The agent would generate visual outputs for quantitative analyses, such as trend lines, anomaly flags, and data distributions, to make results easier to interpret. Structured tests were planned for each query type, with a process to record and classify failure cases by cause. The agent would also interpret the results of data analysis in the context of the original query, and, as a further step, provide recommendations for actions—potentially using domain knowledge or SOP-based retrieval augmented generation (RAG).

To ensure clarity and consistency, the plan included designing report templates for different query types, so that typical reports would merge the data, analysis, and insights into a coherent document. Report generation and visual output handling were to be

implemented to make every analysis run documented and reproducible, supporting the goal of automated, traceable analytics.

4. Accomplishments During the Reporting Period

To date, the core analysis agent was built as a modular Python class (LangGraphReactAnalysisAgent in langgraph_react_analysis_agent.py). The agent is designed to work with different large language models (LLMs) like Anthropic, OpenAI, and Gemini. This is handled through a configuration layer (config.py), making it easy to switch providers and manage API keys without changing the code. This makes it easy to test different models or update credentials as needed.

A major part of the work was connecting the agent to the FastMCP server. This server acts as a central registry for analysis tools. The agent can discover and load these tools at runtime, which means new analysis functions can be added or updated without restarting the system. Here's how the agent loads tools from FastMCP (from the initialize_mcp_tools function):

```
async def initialize_mcp_tools(self):
    fastmcp_server_path = getattr(self.config, 'fastmcp_server_path',
    './fastmcp_server.py')
    abs_server_path = str(Path(fastmcp_server_path).resolve())
    self.mcp_client = MultiServerMCPClient({
        "analysis_server": {
            "command": "python",
            "args": [abs_server_path],
            "transport": "stdio"
        }
    })
    self.tools = await self.mcp_client.get_tools()
    self.react_agent = create_react_agent(self.llm, self.tools,
    checkpoint=checkpointer=self.memory)
```

Figure 1. Code snippet showing how the agent loads tools from FastMCP

On the server side, tools are registered using the FastMCP decorator, for example:

```
# Function: perform_advanced_eda_on_csv, fastmcp_server.py
@mcp.tool()
def perform_advanced_eda_on_csv(file_path: Annotated[str, ...], ... ) ->
str:
    # ...existing code...
    return "\n".join(eda_report)
```

Figure 2. Code snippet showing a tool registered on the FastMCP server

The agent automatically determines the appropriate analysis function for each query and generates both report and visualizations for every run. Session persistence is implemented, so analysis history is maintained and can be reviewed. Basic data cleaning and preparation are handled as part of the analysis pipeline, with FastMCP tools supporting common preprocessing tasks. For example, missing value handling is available as a tool:

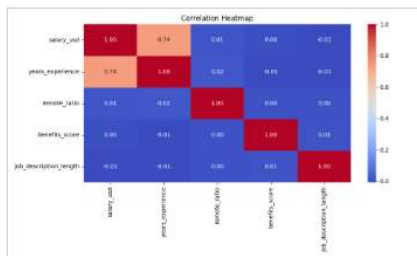
```
# Function: handle_missing_values, fastmcp_server.py
@mcp.tool()
def handle_missing_values(file_path: Annotated[str, ...], strategy:
Annotated[Literal['mean', ...], ...], ...) -> str:
    # ...existing code...
    return "\n".join(report)
```

Figure 3. Code snippet showing an example of a data preprocessing tool

The system then interprets results in the context of the original query. While recommendations for actions are not yet fully automated, the architecture supports future integration of domain knowledge or SOP-based RAG modules. Report templates have been designed, and the agent merges data, analysis, and insights into a single, well-structured output.

2.2 Correlation Analysis

The correlation heatmap visualizes the relationships between key numerical variables.

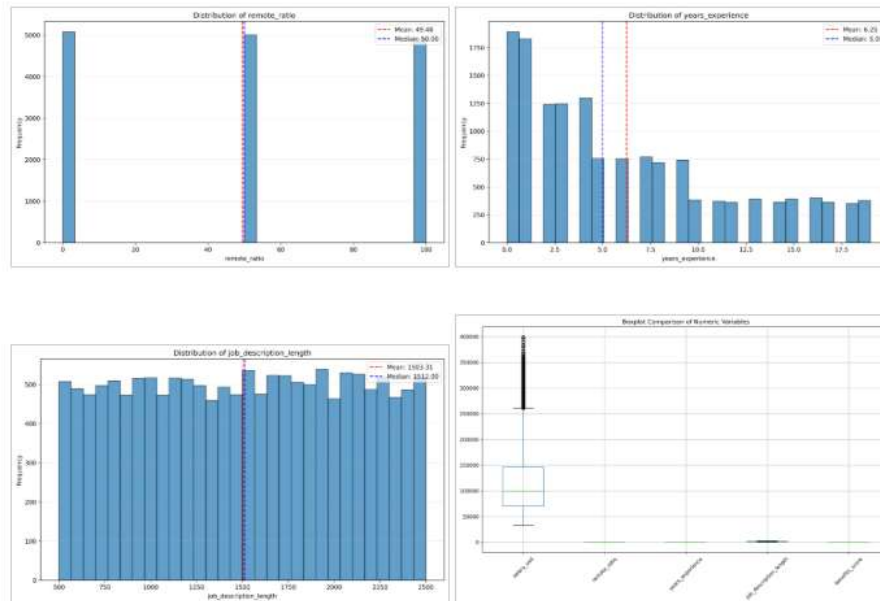


Interpretation: A strong positive correlation is observed between `salary_usd` and `years_experience`, suggesting that experience is a major determinant of salary. The correlation between `salary_usd` and `benefits_score` is also positive, indicating that higher benefits are associated with higher salaries. The negative correlation between `salary_usd` and `remote_ratio` is interesting and warrants further investigation. It might suggest that fully remote positions tend to offer lower salaries compared to on-site or hybrid roles. This could be due to various factors, including cost of living differences across locations or company policies. The relatively weak correlations between other variables suggest that these factors have a less significant impact on salary compared to experience and benefits. Further analysis should explore these relationships in more detail, potentially using regression models to quantify the individual contributions of each variable to salary.

Figure 4. Correlation Analysis. Snippet of generated report.

4. Visual Analysis of Key Variables

Histograms were generated for all numerical variables to visualize their distributions. The boxplots below illustrate the distribution of salaries across different experience levels.



5. Key Insights and Recommendations

- **Experience is paramount:** The strong correlation between salary and experience underscores the importance of experience in determining AI professional compensation. Companies should prioritize candidates with relevant experience when recruiting.
- **Salary distribution:** The right-skewed salary distribution suggests the presence of high-earning individuals, potentially in leadership or specialized roles. Further analysis could segment the data to identify these high-earning roles and their associated characteristics.
- **Targeted recruitment:** Understanding the distribution of experience levels and other factors can inform targeted recruitment strategies. For example, companies seeking entry-level talent should adjust their compensation packages accordingly.

Figure 5. Visuals and Key Insights. Snippet of generated report.

To simulate communication between agents, I implemented a simple structured Agent-to-Agent (A2A) messaging protocol. This uses a dataclass (A2AMessage) to keep messages consistent and easy to track. It allows agents to send instructions, results, and feedback in a standard format.


```
# Test A2A message
test_instruction = A2AMessage(
    id=str(uuid.uuid4()),
    sender_id="orchestrator_test",
    receiver_id=agent.agent_id,
    message_type="instruction",
    content={
        "dataset_path": "ai_job_dataset.csv",
        "instructions": "Perform exploratory data analysis on the
dataset and identify key trends in job market data, provide plots and
tables where appropriate."
    },
    timestamp=datetime.now(),
    session_id=session_id
)

response = await agent.receive_a2a_message(test_instruction)
print(f"Agent response: {response.content}")
```

Figure 6. Code snippet showing the structured Agent-to-Agent (A2A) messaging protocol

Overall, the project is on track, with the core features implemented and tested, and a clear path for further enhancements and integration of advanced capabilities.

5. Current Work

The agent's development is currently focused on refining error handling and ensuring robust session state persistence. Exception management is being improved by standardizing error responses and logging all failures for traceability. For example, the agent uses a dedicated method to generate error messages for the orchestrator:

```
def _create_error_response(self, original_message: A2AMessage, error: str)
-> A2AMessage:
    return A2AMessage(
        id=str(uuid.uuid4()),
        sender_id=self.agent_id,
        receiver_id=original_message.sender_id,
        message_type='error',
        content={
            'error': error,
            'original_message_id': original_message.id
        },
        timestamp=datetime.now(),
        session_id=original_message.session_id
    )
```

Figure 7. Code snippet showing how the agent generates error messages for the orchestrator

Session state is managed using a structured dictionary, ensuring all relevant information (messages, instructions, results, feedback) is tracked and can be recovered if needed:

```
class AgentState(TypedDict):
    messages: List[Any]
    session_id: str
    dataset_path: Optional[str]
    current_instruction: Optional[str]
    analysis_results: List[Dict[str, Any]]
    orchestrator_feedback: Optional[str]
    report_ready: bool
```

Figure 8. Code snippet showing how agent state is managed

Efforts are also underway to enhance report formatting and improve the clarity of data visualizations. The agent saves analysis results and plots to files, making reports more informative and accessible:

```
async def _save_analysis_report(self, analysis_result: str, dataset_path: str, instructions: str):
    # ...existing code...
    report_content = f"""# Data Analysis Report
    **Dataset:** {dataset_path}
    **Generated:** {datetime.now().strftime("%Y-%m-%d %H:%M:%S")}
    ## Analysis Results
    {analysis_result}
    ## Additional Notes
    - All generated plots are saved in the `reportDemo` directory
    """
    with open(report_path, 'w', encoding='utf-8') as f:
        f.write(report_content)
```

Figure 9. Code snippet showing how analysis results and plots are saved

Dataset trials are being conducted to evaluate agent behaviour across different input styles and data complexities, ensuring the agent's generalizability and reliability.

6. Future Plans and Roadmap

Planned enhancements include interactive visual elements such as plots and collapsible sections in reports, leveraging libraries like Plotly or advanced extensions. Persistent session resume features and historical run inspection will allow users to revisit and audit previous analyses, improving traceability and user experience.

The analysis toolset will be expanded by adding more domain-specific functions to the FastMCP server (see `fastmcp_server.py`), supporting use cases beyond generic EDA and statistical analysis. Optimization of LLM fallback and selection heuristics is planned, allowing the agent to dynamically choose the most cost-effective or fastest LLM provider based on workload and latency, thus supporting scalable, enterprise-ready analytics.

7. Background Research and References

Development is guided by official documentation for LangChain, LangGraph, and FastMCP, as well as academic literature on multi-agent orchestration and autonomous analytics. Best practices from enterprise-scale data science pipelines inform the architecture and workflow design. The codebase includes `langgraph_react_analysis_agent.py`, `fastmcp_server.py`, and `config.py`, which collectively implement the agent logic, tool registry, and configuration management.

8. Requirements Gathered

Key requirements include support for multiple LLMs with configurable switching, runtime tool orchestration from a centralized server, session-aware operation with persistent analysis history, automatic Markdown report and plot output, and detailed logging with graceful error recovery. These requirements ensure the agent is flexible, robust, and suitable for real-world deployment.

9. Implementation Details

Configuration management is handled via `.env` and `config.py`, which set up provider credentials and runtime parameters. The agent is designed as a modular class, supporting future extension and integration. Tool discovery is performed in real-time from the FastMCP registry, enabling dynamic workflows. The A2A protocol uses JSON-based structured messages for agent communication:

```
@dataclass
class A2AMessage:
    id: str
    sender_id: str
    receiver_id: str
    message_type: str
    content: Dict[str, Any]
    timestamp: datetime
    session_id: Optional[str] = None
```

Figure 10. Code snippet showing the JSON-based structured messages for agent communication

Report output is managed by saving report and image-based plots to designated directories, ensuring every analysis is documented and reproducible.

10. Testing Approaches and Cases

A comprehensive testing strategy was adopted to ensure the reliability and correctness of the analysis agent.

- **Unit Testing:** Functions such as configuration setup and LLM initialization (see `AgentConfig` and `_initialize_llm` in `langgraph_react_analysis_agent.py`) were tested with various environment settings and API keys to confirm correct provider selection and error handling.
- **Integration Testing:** The agent's ability to load and invoke tools from the FastMCP server was validated by running end-to-end analysis sessions (see `initialize_mcp_tools` in `langgraph_react_analysis_agent.py`). This included verifying tool discovery, execution, and result retrieval.
- **Manual Validation:** Report output and generated visualizations were manually inspected for formatting, completeness, and accuracy (see `_save_analysis_report` and `_save_comprehensive_report` in `langgraph_react_analysis_agent.py`).
- **Functional Testing:** Session persistence and logging consistency were tested by simulating multiple analysis runs, including error scenarios, and confirming that session state and logs were correctly maintained.

11. Technical Challenges and Solutions

One of the main technical challenges was ensuring compatibility with different LLM providers. This was addressed by creating a unified initialization method that works across Anthropic, OpenAI, and Gemini APIs, making it easy to switch providers as

needed. Integrating tools dynamically was another focus; this was solved by using asynchronous FastMCP queries, which allow the agent to load and use new tools in real time. To support reliable output, the system creates and manages directories for reports and session files, and uses file writing methods that handle errors gracefully. Finally, issues with Unicode characters in Windows terminals were fixed to ensure that reports display correctly across different environments.

12. Application of Classroom Knowledge

The project made use of object-oriented programming to keep the code modular and maintainable. Data handling and visualization relied on standard Python libraries like pandas, matplotlib, and seaborn. Configuration management and schema validation were handled with Pydantic, and software testing practices learned in class were applied to ensure the system works as expected.

13. Knowledge Beyond the Classroom

Beyond coursework, this project required learning how to work with the latest LLM APIs and agent orchestration patterns. A custom A2A communication protocol was designed and implemented to support structured agent interactions. The codebase was also developed to be deployment-agnostic, with careful management of API keys and environment variables for security and portability.

14. Other Notable Challenges and Incidents

There were some issues with Unicode handling in Windows terminals, which were resolved to ensure reports display properly. API keys are now managed through environment variables in the .env file to keep deployments secure. Regular discussions with the industry supervisor helped adapt tool formats for FastMCP, making sure the agent can use all available analysis functions effectively.

15. References

- [1] Yao, S., Zhao, J., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." NeurIPS, 2023.
- [2] LangChain. "LangGraph ReAct Agent Template." GitHub, 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [3] LangChain Docs. "Building a LangGraph ReAct Agent." LangChain Documentation, 2024.
- [4] S. Danawale. "How to Generate Dynamic Dashboards and Analytics Using LLMs." Ionio AI Blog, Feb. 2025.
- [5] Ionio-io. "Dynamic Dashboard using LLM." GitHub Repository, 2024.
- [6] Prasanna, D. R. "Introducing the Model Context Protocol." Anthropic Blog, Nov. 2024.
- [7] Microsoft. "Extend Your Agent with Model Context Protocol." Copilot Studio Documentation, 2025.
- [8] Reading-Plus-AI. "MCP-Server-Data-Exploration." GitHub Repository, 2024.
- [9] Zack Saadioui. "Utilize MCP Servers for Real-Time Data Analysis Projects." ArsTurn Blog, 2025.

[10] Surapaneni, R., et al. "Announcing the Agent2Agent Protocol (A2A)." Google Developers Blog, Apr. 2025.

[11] Linux Foundation. "Linux Foundation Launches the Agent2Agent Protocol Project." Press Release, Jun. 2025.

[12] Ran Zhang and Mohannad Elhamod. "Data-to-Dashboard: Multi-Agent LLM Framework for Insightful Visualization in Enterprise Analytics." arXiv:2505.23695, 2025.

END OF DOCUMENT